REAL-TIME MONITORING DASHBOARD

Testing Documentation

HealthTrack: Remote Monitoring for Home-Based Cardiac Rehabilitation

Todd Simonson | MSCS 699 Computer Science Capstone | Michael Solomon | November 15, 2026

This document describes how the dashboard is tested: what each test measures, where a timing starts and stops, the target it is judged against, and how to reproduce the run. The numbers themselves are in `reports/dashboard_test_results.md`, written by `scripts/run_reports.sh` on the machine that ran it.

1. What is tested, and against which requirement

Area	Requirement (Phase 1 matrix)	Target	How
Load time	NFR-09	Initial view within 3 s	Browser, 10 page loads
Real-time updates	FR-18	A posted reading reaches an open dashboard without a reload	Browser, 30 readings
Alert latency	NFR-07	Critical alert on the dashboard within 5 s of ingestion	Browser, 8 alerts
Color-coded abnormal readings	FR-17, NFR-14	Abnormal readings in the alert color with a text label and symbol	Unit tests and browser checks
Worklist ranking	FR-17, NFR-15	Patient with the newest critical alert re-sorts to the top; highest priority reachable in two clicks	Unit tests and browser checks
Data accuracy	FR-17, FR-19	Every displayed value matches the API	Browser checks against direct API calls
Resilience	NFR-11	A dropped connection recovers without a reload	Browser, forced disconnect
Security of the browser session	NFR-03, NFR-04	No token in browser storage or cookies; only the care team's patients	Unit tests; API tests AS-42 to AS-46

2. Unit tests (`tests/`, 47 tests)

Run with `python3 -m pytest`. They need no API or browser.

File	Covers
`test_figures.py`	Reading classification matches the alert engine's rules (context filters, suspect readings), downsampling keeps spikes and every abnormal point, threshold lines and target-zone bands come from the patient's effective thresholds, abnormal markers use the severity color and symbol, session gaps break the line, empty ranges say so, gauges use the patient's limits
`test_ui_logic.py`	Severity badges always carry text and a glyph, worklist cards and patient header content, alert actions offered for each status, folding stream events into the worklist, alert, and gauge state, suspect readings never change the display, one notification pop-up per alert, alert ordering, selection order (deep link, then previous, then top of worklist), the server routes (no ticket when signed out, security headers, logout), tokens kept off the browser, and a concurrent token refresh that rotates exactly once

`dashboard/app.py` callbacks are exercised by the browser tests rather than unit tests, which is why line coverage of `dashboard/` is lower than the alert system's.

3. Browser tests (`scripts/measure_dashboard.py`)

3.1 Test environment

The script builds its own environment so results are repeatable and never touch a working database:

1. A fresh SQLite database in a temporary folder, migrated to head and loaded with the seed data.
2. The alert-system API (`uvicorn`, background workers on) and the dashboard, each on a free local port.
3. Headless Chromium driven by Playwright, at 1500 by 1000 pixels, signed in as Marcus Lee (clinician).
4. Readings are posted to the API as Thomas Reed's heart rate band, exactly as the device app would.

All times come from the same machine's clocks, so the API and the browser share a time base.

3.2 Measurements

Measurement	Starts	Stops
Worklist rendered	Navigation start	First worklist card in the page
Charts drawn	Navigation start	Patient data loaded and all five charts and the heart rate gauge have rendered
Live stream connected	Navigation start	Connection pill reads Live
Reading received	Just before the reading is posted to the API	The browser's WebSocket client receives its event
Reading on the chart	Just before the reading is posted	The live chart update for that reading has been applied
Alert in the alert panel	Just before a breaching pair of readings is posted	A critical alert item appears (observed with a DOM mutation observer)
Worklist re-sorted	Same	The first worklist card carries the critical style
Notification shown	Same	A new critical pop-up appears
Range switch to 7 days	Click	All charts redrawn with the 7-day data
Acknowledge	Click on Acknowledge	The alert's status reads Acknowledged
Reconnect	The test closes the browser's WebSocket	The pill reads Live again (it must pass through Reconnecting)

Each series is reported as n, median (p50), 95th percentile (p95), and maximum. Between alert runs the test resolves Thomas's open alerts through the API and waits for the screen to clear, so each run measures a new alert rather than a repeat folded into an old one.

3.3 Data accuracy and behavior checks

Each check compares what the browser shows with what the API returns at the same moment:

1. Live readings appear on the chart with the posted values.
2. Live readings clear the empty-chart label on a chart that started empty.
3. Worklist order, names, and latest values match `GET /monitoring/worklist` card by card.
4. The heart rate gauge matches `GET /vitals/latest`.
5. The alert panel lists every active alert in scope (`GET /alerts`).

6. Critical alerts carry a text label, not only color.
7. Readings above the ceiling are drawn red, as triangles, labeled Critical.
8. Chart threshold lines equal the patient's effective thresholds (`GET /alert-thresholds`).
9. The 24-hour heart rate chart plots every reading the API returns for that range.
10. Acknowledging in the dashboard records the signed-in clinician in the API.
11. After a dropped connection the stream reconnects and the next reading arrives.

A failed check is reported as FAIL with the observed values; the script does not stop, so one failure does not hide the others.

4. Reproducing the results

From `phase4-dashboard/`, with the alert-system project set up in `../phase4-alert-system/` (its own `.venv`) and Chromium installed for Playwright:

```
source .venv/bin/activate
./scripts/run_reports.sh                       # unit tests with coverage, then the browser tests
python3 -m scripts.measure_dashboard --runs 20 --readings 50 --alerts 10    # larger samples
python3 -m scripts.measure_dashboard --headed  # watch the browser while it runs
```

Outputs: `reports/dashboard_test_results.md` (tables), `reports/dashboard_e2e.json` (every raw timing), `reports/pytest-unit.txt`, `reports/junit-unit.xml`, and screenshots in `reports/screens/`.

5. Limits of these results

- Everything runs on one machine, so network latency between a clinic and a server is not included. The 3 s and 5 s targets leave room for it, but a deployed test over a real network is still needed (Phase 6).
- One browser client is measured. Many simultaneous dashboards are measured at the broker level in the alert system's performance report (WebSocket fan-out), not in a browser.
- The database is SQLite. PostgreSQL behavior is covered by the alert system's own test suite.
- Contrast was computed from the palette values rather than with an automated audit tool: every severity text color is at least 4.6:1 on white and on its own tint (WCAG 2.1 AA needs 4.5:1), and chart symbols are at least 4.9:1 (3:1 needed).

6. Results from the development machine

Run 2026-11-12T22:58:53+00:00 on macOS-26.5.1-arm64-arm-64bit (arm64, 8 cores), Python 3.12.13, chromium 153.0.8010.12 headless. The API and dashboard ran locally on this machine with the seed data; times are from this machine's clock.

Unit tests

47 unit tests, 47 passed, 0 failed (0.90 s): figure builders, reading classification, gauges, components, stream state updates, and the dashboard server routes. Line coverage of `dashboard/`: 71.1%.

1. Load time (requirement NFR-09: initial view within 3 s)

Measured from navigation start in the browser, signed in, across 10 page loads.

Measurement	n	p50 (ms)	p95 (ms)	max (ms)
Worklist rendered	10	152.6	596.6	596.6
Charts drawn	10	674.8	762.3	762.3
Live stream connected	10	691.0	774.1	774.1

Charts drawn p95 762.3 ms: target met. Each load transferred about 98 KB, because the Dash and Plotly bundles come from the browser cache after the sign-in page.

2. Real-time updates

A reading posted to the API (as the patient's device) until it reaches the browser and is drawn on the open chart.

Measurement	n	p50 (ms)	p95 (ms)	max (ms)
Reading received over the WebSocket	30	12.4	15.5	288.1
Reading appended to the chart	30	122.3	131.3	393.1

A breaching reading pair posted until each part of the screen shows the new critical alert (requirement NFR-07: within 5 s).

Measurement	n	p50 (ms)	p95 (ms)	max (ms)
Alert in the alert panel	8	254.6	269.1	269.1
Worklist re-sorted, patient first	8	250.6	265.1	265.1
Notification toast shown	8	258.1	274.1	274.1

Alert panel p95 269.1 ms: target met.

3. Interaction timing

- Switch the charts to 7 days (reload and redraw): 18.3 ms
- Acknowledge an alert, click to updated status: 62.0 ms
- Dropped WebSocket to live again (automatic reconnect): 1042.1 ms

4. Data accuracy and behavior checks (11 of 11 passed)

Check	Result	Detail
Live readings appear on the chart with the posted values	PASS	last three plotted [107, 108, 109], posted [107, 108, 109]
Live readings clear the empty-chart label on a chart that started empty	PASS	chart labels now ['Target zone 95-115', 'Exercise ceiling 128', 'Resting limit 90']
Worklist order, names, and latest values match the API	PASS	2 cards compared field by field
Heart rate gauge matches the latest valid reading	PASS	gauge 138, api 138
Alert panel shows every active alert in scope	PASS	panel 2, api 2
Critical alerts carry a text label, not only color	PASS	badge '▲ CRITICAL'
Readings above the ceiling are drawn red, as triangles, labeled Critical	PASS	{'color': '#B42318', 'symbol': 'triangle-up', 'text': 'Critical reading, exercise'}

Check	Result	Detail
Chart threshold lines come from the patient's effective thresholds	PASS	ceiling 128.0, resting 90.0, drawn [90, 95, 128]
24-hour heart rate chart plots every reading the API returns	PASS	chart 46, api 46
Acknowledging in the dashboard records the clinician in the API	PASS	acknowledged_by 00000000-0000-4000-a000-000000000002
After a dropped connection the stream reconnects and live updates resume	PASS	live again after 1042 ms; next reading arrived

5. Page loads

Run	DOM ready (ms)	Worklist (ms)	Charts (ms)	Live (ms)	Transferred (KB)
1	77.5	160.9	690.3	712.8	98.4
2	68.8	153.5	677.3	690.7	98.4
3	71	152.7	675.8	690.1	98.4
4	69.7	153.6	673.8	691.9	98.4
5	70.4	152.4	680	707	98.4
6	68.3	596.6	762.3	774.1	98.4
7	70.3	152.4	667.9	691.2	98.4
8	68.5	152.6	670.1	690.3	98.4
9	65.4	143.7	653.2	674.1	98.4
10	65.8	146.3	670.9	690.1	98.4

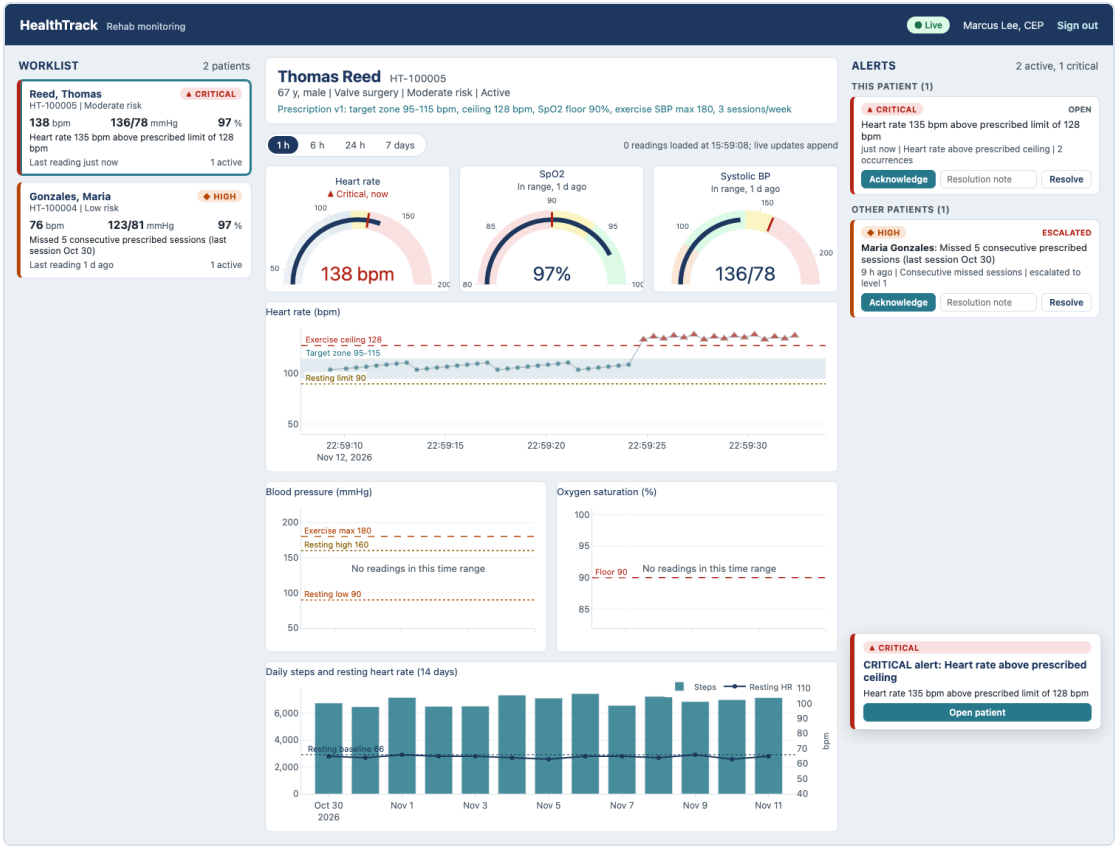


Figure 1. Screen captured by the browser test after a critical heart rate alert (test run of November 12, 2026).

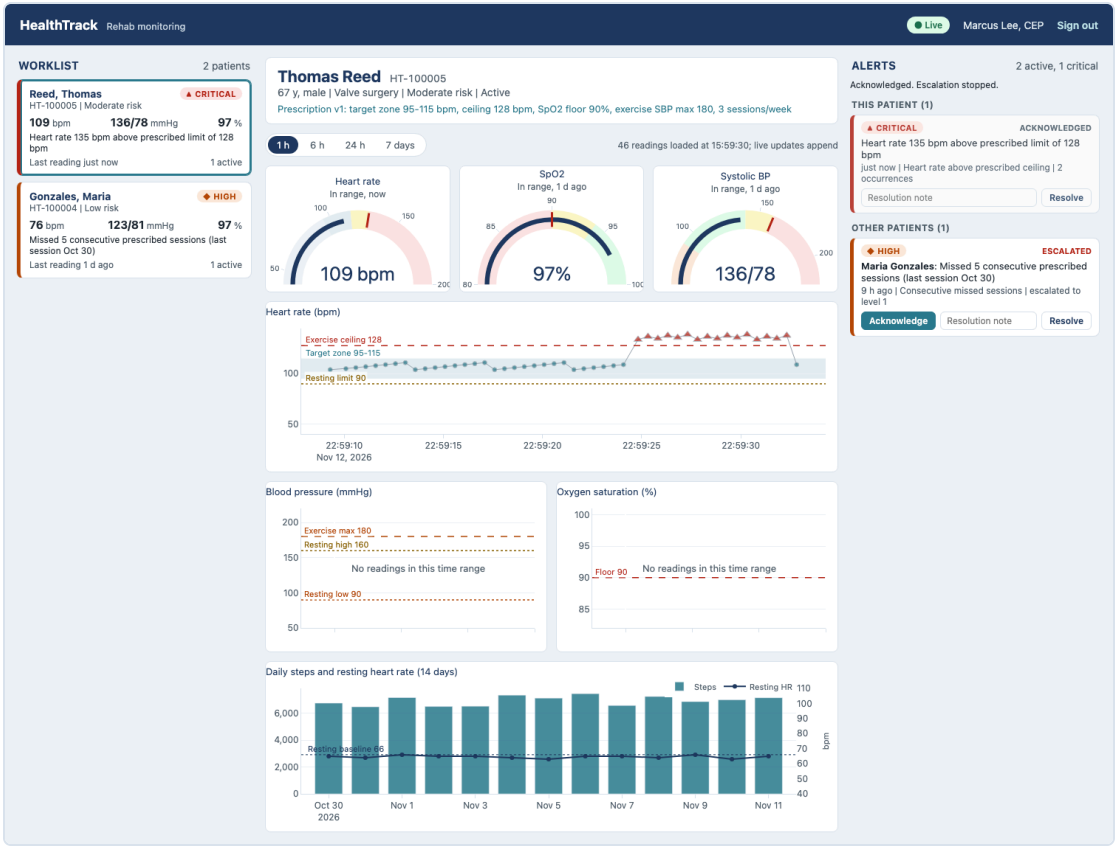


Figure 2. The same screen after the test acknowledged the alert from the dashboard.